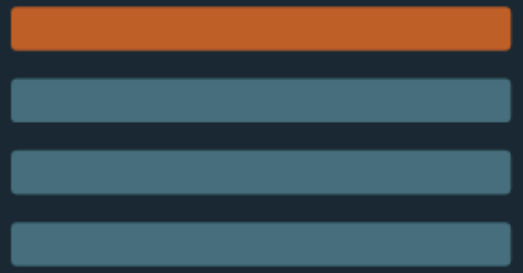
MAESTRÍA · ESPACIO ACADÉMICO

Gestión de Configuración y Mantenimiento de Software

Sesión 2 — Unidad 2 · Gestión de configuración del software

El orden que permite que varios trabajen sobre lo mismo sin destruirlo.



César Augusto Vega Fernández

Encuentro sincrónico · Ítems de configuración · Líneas base · Control de cambios · Trazabilidad · IEEE 828

EL RETO DE LA UNIDAD 2 · ACTIVIDAD EN LA PLATAFORMA

Saber en todo momento qué hay, qué cambió y quién lo aprobó

«Ya sabemos qué le pasa al sistema. Lo que no sabemos es qué versión está en producción, qué cambió desde la anterior ni quién lo aprobó.

Necesitamos una estrategia de gestión de configuración que podamos aplicar desde la próxima entrega.»

Responsable del sistema que eligió cada grupo

Actividad 2 · Diseño de estrategia de gestión de configuración

· 20 % · 16 horas · grupal, de 2 a 3

· Documento técnico de 8 a 10 páginas o README.md

· Ítems de configuración · líneas base · control de cambios

· Trazabilidad · roles y responsabilidades · modelo de gestión de configuración

Coherencia con la Unidad 1

· Los riesgos de cambios no controlados del F-06 son lo que la estrategia tiene que resolver. La Unidad 3 la implementa en Git y en un pipeline.

Una estrategia que no responde a los riesgos del diagnóstico es un procedimiento genérico. La rúbrica pide coherencia con la actividad anterior.

Los formatos de esta unidad son del F-08 al F-14: la numeración continúa la de la Unidad 1. No se suben a la plataforma.

CÓMO VAMOS A USAR LA SESIÓN

Primero qué significa, después ejemplos, después ustedes

PRIMERO

Conceptos

Ítems de configuración, identificación, líneas base, control de cambios, trazabilidad y roles.

DESPUÉS

Ejemplos

Cómo gestiona Flask su configuración, leído en su propio repositorio: qué controla, cómo publica y qué se puede rastrear.

Y EL RESTO DE LA SESIÓN

El hacer

El taller en salas: el inventario de ítems de configuración del sistema propio y su primera línea base.

El bloque que no se recorta es el tercero. Los ejemplos están completos en el recorrido resuelto del repositorio.

BLOQUE 1 · EL PROBLEMA

Cuando coordinar se vuelve un problema de ingeniería

Un caso de banca

Un defecto llega a producción sin que nadie lo detecte, y los reportes de conciliación que un banco genera dejan de funcionar a nivel nacional. Unas horas sin reportes confiables afectan a otras entidades. El código no es lo que falla: falta control sobre qué versión llegó a producción y en qué condiciones se validó.

Sin gestión de configuración

Conflictos de integración inesperados.

Regresiones que nadie sabe explicar.

Nadie puede decir con certeza qué versión corre en producción.

Con gestión de configuración

Se comparten productos de trabajo con reglas claras.

Las decisiones técnicas quedan registradas.

Varias personas cambian la misma base sin romper la estabilidad.

Berczuk y Appleton (2002) describen la gestión de configuración como el «pegamento» que une artefactos, cambios y personas de un proyecto.

BLOQUE 1 · DE QUÉ SE COMPONE

Las actividades de la gestión de configuración

Planificación F-13	Quién hace qué, con qué herramientas y con qué reglas. Es el plan de gestión de configuración.
Identificación F-08 · F-09	Qué elementos se controlan, cómo se nombran y cómo se distinguen sus versiones.
Control de cambios F-10 · F-11	Cómo se propone, evalúa, aprueba e incorpora un cambio sobre una línea base.
Registro del estado F-12	Qué versión de cada elemento está en cada línea base y qué cambios la produjeron.
Auditoría F-14	Verificar que lo entregado coincide con lo aprobado y con lo registrado.

Actividades de la gestión de configuración según IEEE 828-2012, Standard for Configuration Management in Systems and Software Engineering.

BLOQUE 1 · IDENTIFICACIÓN

Ítems de configuración: qué merece controlarse

Código fuente
El producto y sus pruebas.

Configuración
Archivos de configuración de la aplicación y de los entornos.

Construcción y dependencias
Manifiestos, archivos de bloqueo, scripts de construcción.

Integración y despliegue
Pipelines, scripts de despliegue, infraestructura como código.

Documentación técnica
Arquitectura, decisiones, manuales de operación.

Modelos y datos de referencia
Esquemas de base de datos, migraciones, catálogos.

Gobierno del proyecto
Plantillas de cambio, registro de cambios, licencia.

Artefactos entregados
Paquetes e imágenes construidos, con su identificador.

Un ítem de configuración es cualquier artefacto cuyo cambio puede afectar el comportamiento o la reproducibilidad del sistema. Un archivo de configuración puede tumbar producción sin que el código cambie.

Humble y Farley (2010): todo lo necesario para construir, desplegar y operar el sistema debe estar bajo control de versiones.

BLOQUE 1 · IDENTIFICACIÓN

Cómo se nombra y se distingue cada versión

Qué se define	Pregunta que responde	Cómo se ve en Git
Identificador del ítem	¿Cómo se refiere el equipo a este elemento sin ambigüedad?	Ruta en el repositorio, nombre del módulo o del artefacto
Esquema de versiones	¿Qué número lleva cada entrega y qué significa?	Etiquetas: 1.4.0, 1.4.1
Ramas permanentes	¿Dónde se desarrolla y dónde se mantiene lo ya entregado?	main, una rama de mantenimiento
Versión declarada	¿El artefacto dice qué versión es?	La versión del manifiesto coincide con la etiqueta
Identificador del artefacto	¿Cómo se comprueba que el paquete entregado es el construido?	Hash del paquete o de la imagen

El esquema de versiones se profundiza en la Unidad 3 con el versionamiento semántico. Aquí basta con que sea explícito y consistente.

BLOQUE 1 · LÍNEAS BASE

Una línea base es un compromiso, no una fotografía

Qué es

Un conjunto de ítems de configuración en un estado acordado, revisado y aprobado, que sirve como punto de referencia estable.

A partir de ella, todo cambio pasa por el control de cambios.

Cómo se reconoce una buena línea base

Tiene un nombre y una fecha.

Dice qué ítems incluye y en qué versión.

Alguien la aprobó y se sabe quién.

Se puede reconstruir: otra persona obtiene exactamente lo mismo.

Sin líneas base el equipo no sabe qué versión está en producción ni a qué punto volver si algo sale mal.

BLOQUE 1 · CONTROL DE CAMBIOS

Gobernanza sin burocracia

1. **Solicitud**
   Qué se quiere cambiar, por qué y qué ítems toca.
2. **Análisis de impacto**
   Qué más se afecta: otros ítems, líneas base, usuarios, riesgos.
3. **Decisión**
   Aprobado, rechazado o aplazado, por quien tiene la autoridad para ese tipo de cambio.
4. **Implementación y verificación**
   El cambio se hace sobre la línea base y se prueba.
5. **Registro y cierre**
   Queda el vínculo entre la solicitud, el cambio y la nueva versión.

Proporcional al riesgo

Un cambio de bajo riesgo puede aprobarse con la revisión de un par y las pruebas automáticas.

Un cambio que toca una línea base de producción necesita más.

La tensión real es entre control bien diseñado y burocracia mal diseñada.

BLOQUE 1 · TRAZABILIDAD

Poder responder preguntas difíciles

Requisito o issue
Por qué se cambió

›

Solicitud o PR
Qué se aprobó

›

Commits
Qué se modificó

›

Versión
Dónde quedó

›

Prueba
Cómo se verificó

Pregunta	Qué la responde
¿Qué cambió entre la versión 2.3 y la 2.4?	El historial entre las dos etiquetas y el registro de cambios
¿Qué requisito motivó esta modificación?	La referencia del commit o del PR al issue
¿Qué artefactos afectó este defecto?	Los ítems tocados por los cambios que lo corrigieron

En entornos regulados la trazabilidad es obligatoria: PCI-DSS en pagos con tarjeta, HIPAA en salud, SOX en reportes financieros.

BLOQUE 1 · QUIÉN HACE QUÉ

Roles y el plan de gestión de configuración

Rol	Responsabilidad
Responsable de configuración	Mantiene el inventario, las líneas base y el registro del estado
Autoridad de cambios	Decide sobre las solicitudes según su riesgo; puede ser una persona o un comité
Desarrolladores	Proponen e implementan cambios siguiendo el proceso
Responsable de entregas	Construye, identifica y publica cada versión a partir de una línea base

En un equipo de 2 o 3

Una persona puede tener varios roles.

Lo que no puede pasar es que un rol no tenga dueño.

El plan dice quién tiene cada uno.

BLOQUE 1 · LA CADENA QUE SOSTIENE EL DOCUMENTO

Del inventario a la trazabilidad

Inventario
F-08 · qué se controla

›

Identificación
F-09 · cómo se nombra y versiona

›

Líneas base
F-10 · puntos de referencia

›

Control de cambios
F-11 · cómo entra un cambio

›

Trazabilidad
F-12 · cómo se rastrea

El F-13 reúne todo en el modelo de gestión de configuración, con los roles. El F-14 verifica antes de entregar.

Qué controla Flask en su repositorio

Categoría	Archivos	Ítems clave
Documentación	88	docs/ completo
Pruebas	61	tests/
Ejemplos	42	examples/
Código fuente	24	src/flask/
Gobierno del proyecto	7	CHANGES.rst, plantillas de issue y de PR, licencia
Entorno y herramientas	6	.pre-commit-config.yaml, .devcontainer/, .readthedocs.yaml
Integración continua	4	.github/workflows/: pruebas, publicación, pre-commit, bloqueo
Construcción y dependencias	2	pyproject.toml y uv.lock

Lo que no está en el repositorio

Los paquetes publicados en PyPI.

Los issues, los PR y sus vínculos.

Quién puede fusionar y quién puede publicar.

También son configuración: el inventario tiene que nombrarlos.

python herramientas/configuracion.py inventario flask --ref 3.1.3 · 235 archivos versionados en la etiqueta 3.1.3.

BLOQUE 2 · EJEMPLO GUIADO 2 · F-09 Y F-10

Identificación y líneas base en Flask

Etiquetas identificación	Cada versión es una etiqueta X.Y.Z sin prefijo: 3.1.0, 3.1.1, 3.1.2, 3.1.3.
Versión declarada identificación	En la etiqueta 3.1.3, pyproject.toml declara **3.1.3**. En main declara **3.2.0.dev**: se ve que es desarrollo.
Ramas identificación	**main** para la siguiente versión, **stable** para las correcciones de la vigente. stable se integra a main con frecuencia.
Línea base de entrega línea base	Al subir una etiqueta, el flujo de publicación construye el paquete, crea el borrador de la entrega en GitHub y publica en PyPI.
Construcción reproducible línea base	Fija la fecha del paquete con la del último commit (SOURCE_DATE_EPOCH): construir dos veces la misma etiqueta da el mismo paquete.

Evidencia: git tag, pyproject.toml en cada etiqueta y .github/workflows/publish.yaml.

BLOQUE 2 · EJEMPLO GUIADO 3 · F-11

Cómo entra un cambio en Flask

1. **Issue primero**
   La plantilla de PR pide abrir antes un ticket, salvo erratas de documentación.
2. **PR con lista de verificación**
   Pruebas que fallen sin el cambio, documentación, entrada en CHANGES.rst con el vínculo al issue.
3. **Pruebas automáticas en cada PR**
   En la versión 3.1.3, 12 combinaciones de versión de Python, sistema operativo y versiones de dependencias, más verificación de tipos.
4. **Integración**
   El cambio se fusiona en main, o en stable si es una corrección de la versión vigente.
5. **Entrega**
   Una etiqueta dispara la publicación, que corre en un entorno de GitHub llamado publish.

Integridad del pipeline

Las 20 acciones externas de sus 4 flujos de CI están fijadas por hash de commit, no por etiqueta.

Las pruebas corren con las dependencias bloqueadas en uv.lock.

El pipeline también es un ítem de configuración.

Evidencia: .github/pull_request_template.md, .github/workflows/tests.yaml y publish.yaml, uv.lock.

¿Se puede rastrear la versión 3.1.0?

Desde el registro de cambios de la 3.1.0	Resultado
Entradas nuevas con referencia a PR o issue	10 de 10
Entradas que se siguen hasta un commit solo con Git	2: las que citan los PR 5623, 5624 y 5633
Entradas que se siguen buscando el contenido del cambio	8 más: se llega al commit, y en 7 también al PR; el vínculo entre issue y PR está solo en GitHub
Merges con número de PR en el mensaje	51 de 70
Commits individuales con referencia	3 de 93

La lección

La trazabilidad de Flask existe, pero buena parte vive en la plataforma.

Si el proyecto cambiara de plataforma, perdería esos vínculos.

Su estrategia debe decir dónde vive cada vínculo.

python herramientas/configuracion.py traza flask --desde 3.0.3 --hasta 3.1.0 --registro CHANGES.rst

Lo que el repositorio no dice

Visible en el repositorio	Solo en la configuración de las plataformas
Qué se exige a un cambio: la plantilla de PR.	Quién puede fusionar en main y en stable.
Qué se prueba y con qué dependencias: los flujos de CI y uv.lock.	Quién aprueba una publicación en el entorno publish.
Cómo se construye una entrega: publish.yaml.	Quién administra el paquete en PyPI.

Para su sistema: escriban los roles en el repositorio, por ejemplo en el README o en un archivo CODEOWNERS. Un rol que solo existe en la configuración de una plataforma no se audita.

EL HACER · TALLER EN SALAS

De la sesión salen con el inventario y la primera línea base

1	**Correr el inventario** Sobre el repositorio del sistema, o listar a mano los artefactos si aún no está en Git.	5 min
2	**Decidir qué es ítem de configuración** Con el criterio: ¿su cambio afecta el comportamiento o la reproducibilidad?	10 min
3	**Nombrar lo que vive fuera del repositorio** Paquetes, secretos, configuración de plataformas, bases de datos.	5 min
4	**Definir la primera línea base** Nombre, ítems incluidos, quién la aprueba y cómo se marca.	7 min
5	**Conectar con el F-06** Qué riesgo de la Unidad 1 resuelve cada decisión.	3 min

Al salir de la sala: el F-08 en borrador y la primera fila del F-10.

PARA TENER AL LADO DURANTE EL TALLER

Comandos y preguntas

Comandos

python herramientas/configuracion.py inventario sistema

python herramientas/configuracion.py traza sistema **--desde** v1 **--hasta** v2

git tag -a v1.0.0 **-m** "Línea base de entrega"

git ls-files para ver qué está versionado

Preguntas para cada artefacto

¿Si cambia, cambia el comportamiento del sistema?

¿Sin él, se puede reconstruir el sistema?

¿Quién es su responsable?

¿En qué línea base entra?

CIERRE · QUÉ SIGUE

De diseñar el control a implementarlo

ESTA SEMANA

Actividad 2

Completar el inventario, las líneas base, el proceso de control de cambios, la trazabilidad y los roles. Verificar con el F-14 antes de entregar.

UNIDAD 3

Control de versiones y automatización

La estrategia se implementa: repositorio en Git, estrategia de ramas, versionamiento semántico y un pipeline de CI/CD funcionando.

Lo que se diseña en esta unidad es lo que se implementa en la siguiente. Una estrategia que no se pueda implementar en Git no sirve para la Unidad 3.

REFERENCIAS

Fuentes de la sesión

Berczuk, S., & Appleton, B. (2002). Software configuration management patterns: Effective teamwork, practical integration. Addison-Wesley.

Humble, J., & Farley, D. (2010). Continuous delivery: Reliable software releases through build, test, and deployment automation. Addison-Wesley.

IEEE. (2012). IEEE Standard for configuration management in systems and software engineering (IEEE Std 828-2012). IEEE.

Kim, G., Humble, J., Debois, P., & Willis, J. (2016). The DevOps handbook. IT Revolution Press.

Chacon, S., & Straub, B. (2014). Pro Git (2.ª ed.). Apress. https://git-scm.com/book

Pallets. (2026). Flask [Repositorio de código fuente]. https://github.com/pallets/flask